Dynamic Programming

Kanishk Goel

24 February, 2026

1 Maximum Weight Independent Set on a Path Graph

Independent Set: A subset of vertices $S \subseteq V$ such that no two vertices in S are adjacent. For path graph, this would mean if $v_i \in S \implies \{v_{i-1}, v_{i+1}\} \cap S = \emptyset$

Problem: MWIS in path graph
Input: An path graph $G = (V, E)$ with n vertices, denoted $v_1, v_2, \ldots, v_n$, each having non negative weight w_i
Output: Find an independent set where sum of weights is maximised, also find its sum

Let $A[i]$ be the maximum weight for the first i vertices. The Recursion becomes:

$$A[i] = max(A[i-1], A[i-2] + w_i)$$

Solution
$A[0] \leftarrow 0, A[1] \leftarrow w[1]$ **for** $i \leftarrow 2$ *to* n **do** $\quad A[i] = max(A[i-1], A[i-2] + w_i)$ **return** $A[n]$

Go backward from n to reconstruct the set of vertices chosen.

2 Weighted Interval Scheduling

Problem: Weighted Interval Scheduling
Input: set of jobs, with each job having a start time, finish time, and a weight (value/profit).
Output: A subset of non-overlapping jobs that maximizes the total weight.

- Let the n intervals be sorted by finish times.
- $OPT[i]$ be the maximum weight for the first i intervals or jobs.
- $p(i)$ be the largest index $j < i$ such that $f_j \leq s_i$.

The Recursion becomes:

$$OPT[i] = max(OPT[i-1], v_i + OPT[p[(i)])$$

Solution
$OPT[0] \leftarrow 0$ //of size n+1 **for** $i \leftarrow 1$ *to* n **do** $\mid$ $OPT[i] = max(OPT[i-1], v_i + OPT[p[(i)])$ **return** $OPT[n]$

Here, the values $p(i)$ have been precomputed using binary search, and the sorting has been done beforehand.

3 0/1 Knapsack Problem

Problem: 0/1 Knapsack
Input: There are n items with weight w_i, value v_i and W is the maximum weight that the knapsack can hold. **Output:** Find the maximum value that the knapsack can hold.

Let $K[i, w]$ be the maximum value achievable using the first i items with a combined weight of w_i. The Recursion becomes:

$$K[i,w] = \begin{cases} K[i-1,w] & \text{if } w_i > w, \\ \max\left(K[i-1,w],\ v_i + K[i-1, w-w_i]\right) & \text{if } w_i \leq w. \end{cases}$$

Here $w \in [0, W]$ and K is a $2D$ array of size $(n+1) \times (W+1)$

Solution

```
for w ← 0 to W do
|  K[0, w] ← 0

for i ← 0 to n do
|  K[i, 0] ← 0

for i ← 1 to n do
|  for j ← 1 to W do
|  |  if w[i] > j then
|  |  |  K[i, j] ← K[i − 1, j]
|  |  else
|  |  |  K[i, j] ← max(K[i − 1, j], v[i] + K[i − 1, j − w[i]])
return K[n, W]
```

4 Longest Increasing Subsequence

Problem: LIS

Input: Array a,with n numbers
Output: Find the longest strictly increasing subsequence in a

4.1 Solution in $O(n^2)$

Define $d[0, 1, \ldots n]$, where $d[i]$ is the length of the longest increasing subsequence ending at index i. Therefore,

$$d[i] = \max\left(1, \max_{\substack{j<i \\ a[j]<a[i]}} (d[j] + 1)\right)$$

maintaining an array $p[i] = j$, where j is the maximal value of $d[j]$ found for index i, also allows us to reconstruct the sequence (Not required but is the easier way).

4.2 Solution in $O(n \log n)$

We define $d[0, \ldots n]$ such that $d[l]$ defines the smallest element at which an increasing subsequence of length l ends.

We go from left to right and find the place to insert $a[i]$ in d using binary search. Hence, the Time Complexity - $O(n \log n)$ and Space Complexity - $O(n)$.

4.2.1 Restoring subsequence

We maintain two auxiliary arrays, one that tells us the index of elements in $d[]$, and an array of ancestors $p[]$ such that $p[i]$ will be the index of the previous element for the optimal subsequence ending at element i.

5 Coin Change (Ways)

Problem: Coin Change (Ways)
Input: Denominations of coins in array A and the target amount N **Output:** Number of ways to achieve the target amount using denomination of coins such that the order does not matter.

5.1 Solution in $O(N \cdot K)$

Idea is simple, consider a $dp[i][j]$ such that it stores the number of ways to achieve value i using the last coin as $A[x]$ where $x < j \wedge A[x] < A[j]$. Since, the order does not matter, we are counting only the lexicographical order. Now, from coin j, we can either use the coin j again, or move onto obtaining the value i using coin $j + 1$.

5.2 Transition

$$dp[i, j] = dp[i, j - 1] + dp[i - A[j], j]$$

6 Subset Sum with Add and Erase

Problem: Subset Sum
Input: An array A and value K **Output:** Number of subsets of A with sum K

Subset: Set of indices (not values).

6.1 Solution in $O(N \cdot K)$

$dp[i]$ = Number of subsets whose sum is i. Base Case: $dp[0] = 1$ (the empty subset).

6.2 Transition

$$dp[i] = dp[i] + dp[i - x] \quad \forall x \leq i$$

6.3 Add and Erase

Start with an empty array, and follow Q queries of form either $+x$ or $-x$.

6.3.1 Addition

For a query $+x$, transitions becomes:

$$\forall K, K-1 \dots x, \text{ in this order do:} \quad dp[i] = dp[i] + dp[i-x]$$

6.3.2 Erasing

For a query $-x$, transitions becomes:

$$\forall x, x+1 \dots K, \text{ in this order do:} \quad dp[i] = dp[i] - dp[i-x]$$

7 Levenshtein Distance

Problem: Levenshtein Distance	
Input:	Two strings x and y
Output:	Minimum number of edits required to convert x into y, operations allowed are insert (in x), remove (from x) and modify (a character of x) which cost of each operation being 1.

Idea is simple, suppose $dp[a, b]$ denotes the minimum number of edits required to match $x[0, \dots a]$ and $y[0, \dots b]$

$$dp[a, b] = \min\left(dp[a-1, b] + 1, dp[a, b-1] + 1, dp[a-1, b-1] + cost(a, b)\right)$$

where $cost(a, b)$ is defined as:

$$cost(a, b) = \begin{cases} x[a] = y[b] & 0, \\ x[a] \neq y[b] & 1 \end{cases}$$

Here, matching of string till a and b means either inserting at position a ($dp[a, b-1]$), removing at position a ($dp[a-1, b]$) or modifying (might not need to if $x[a] = y[b]$) at position a ($dp[a-1, b-1]$).